



# HUST

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.





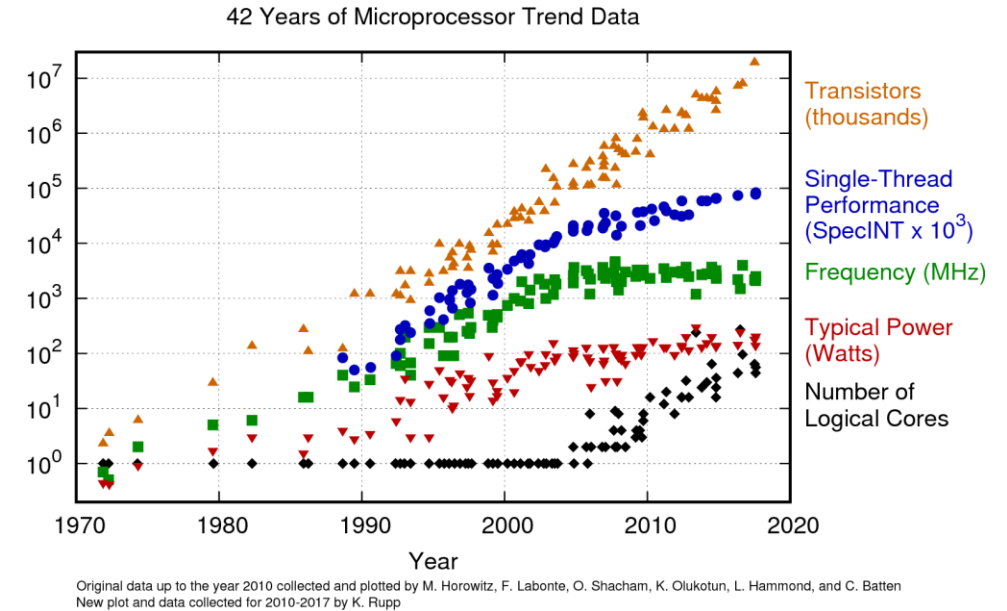
**ĐẠI HỌC  
BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY  
OF SCIENCE AND TECHNOLOGY

# Parallelism

ONE LOVE. ONE FUTURE.

# How to speeding up your programs

- The simplest way: **increasing the clock speed**
- **Limitation:**
  - the computations do take some amount of time to perform. If you raise the clock speed too high, the chip starts performing the next computation before the previous computation is complete, and could be using garbage inputs.
  - as the clock speed increases, so does the power used by the chip. If you raise the clock speed too high, the chip could melt.
- Another way: computing multiple instructions at the same time (**in parallel**)



# Types of parallelism

- Instruction level parallelism
- Single-instruction multiple-data parallelism
- Multi-thread parallelism
- Distributed computing
- Accelerated computing (GPU, TPU, FPGA,...)

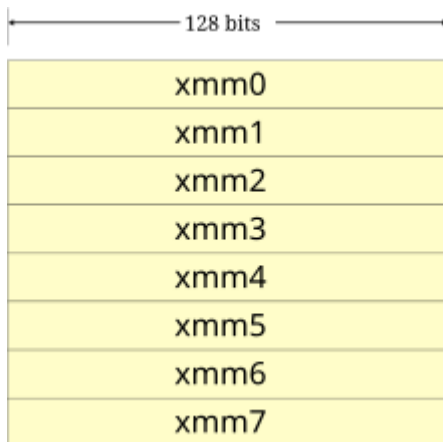
# Instruction level parallelism (ILP)

- Allow multiple instructions executed in parallel if they do not depend on each other
  - Pipeline
- **Limitation:**
  - there is a finite amount of instruction-level parallelism available within any section of a program, so there's an upper limit on how fast ILP can make our programs.
  - rely on the optimization capability of compilers

# Single-instruction multiple-data parallelism (SIMD)

- Using specialized instructions to perform multiple operations at the same time
  - Vector instructions
  - Intel x86: MMX, SSE, SSE2, SSE4, AVX
  - AMD: 3DNow
  - Sparc: VIS
  - ARM: Neon

# Intel x86's SSE



Registers

## Floating-point instructions [\[edit\]](#)

- Memory-to-register/register-to-memory/register-to-register data movement
  - Scalar – `MOVSS`
  - Packed – `MOVAPS`, `MOVUPS`, `MOVLPS`, `MOVHPS`, `MOVLHPS`, `MOVHLPS`, `MOVMSKPS`
- Arithmetic
  - Scalar – `ADDSS`, `SUBSS`, `MULSS`, `DIVSS`, `RCPS`, `SQRTSS`, `MAXSS`, `MINSS`, `RSQRTSS`
  - Packed – `ADDPS`, `SUBPS`, `MULPS`, `DIVPS`, `RCPPS`, `SQRTPS`, `MAXPS`, `MINPS`, `RSQRTPS`
- Compare
  - Scalar – `CMPSS`, `COMISS`, `UCOMISS`
  - Packed – `CMPPS`
- Data shuffle and unpacking
  - Packed – `SHUFPS`, `UNPCKHPS`, `UNPCKLPS`
- Data-type conversion
  - Scalar – `CVTSS2SS`, `CVTSS2SI`, `CVTTSS2SI`
  - Packed – `CVTPI2PS`, `CVTPS2PI`, `CVTTPS2PI`
- Bitwise logical operations
  - Packed – `ANDPS`, `ORPS`, `XORPS`, `ANDNPS`

## Integer instructions [\[edit\]](#)

- Arithmetic
  - `PMULHUW`, `PSADBW`, `PAVGB`, `PAVGW`, `PMAXUB`, `PMINUB`, `PMAWS`, `PMINSW`
- Data movement
  - `PEXTRW`, `PINSRW`
- Other
  - `PMOVBMSKB`, `PSHUFW`

## Other instructions [\[edit\]](#)

- `MXCSR` management
  - `LDMXCSR`, `STMXCSR`
- Cache and Memory management
  - `MOVNTQ`, `MOVNTPS`, `MASKMOVQ`, `PREFETCH0`, `PREFETCH1`, `PREFETCH2`, `PREFETCHNTA`, `SFENCE`

Instructions

```
movaps xmm0, [v1] ;xmm0 = v1.w | v1.z | v1.y | v1.x
addps xmm0, [v2] ;xmm0 = v1.w+v2.w | v1.z+v2.z | v1.y+v2.y | v1.x+v2.x
movaps [vec_res], xmm0 ;xmm0
```

Example



# Intel x86's SSE : matrix multiplication

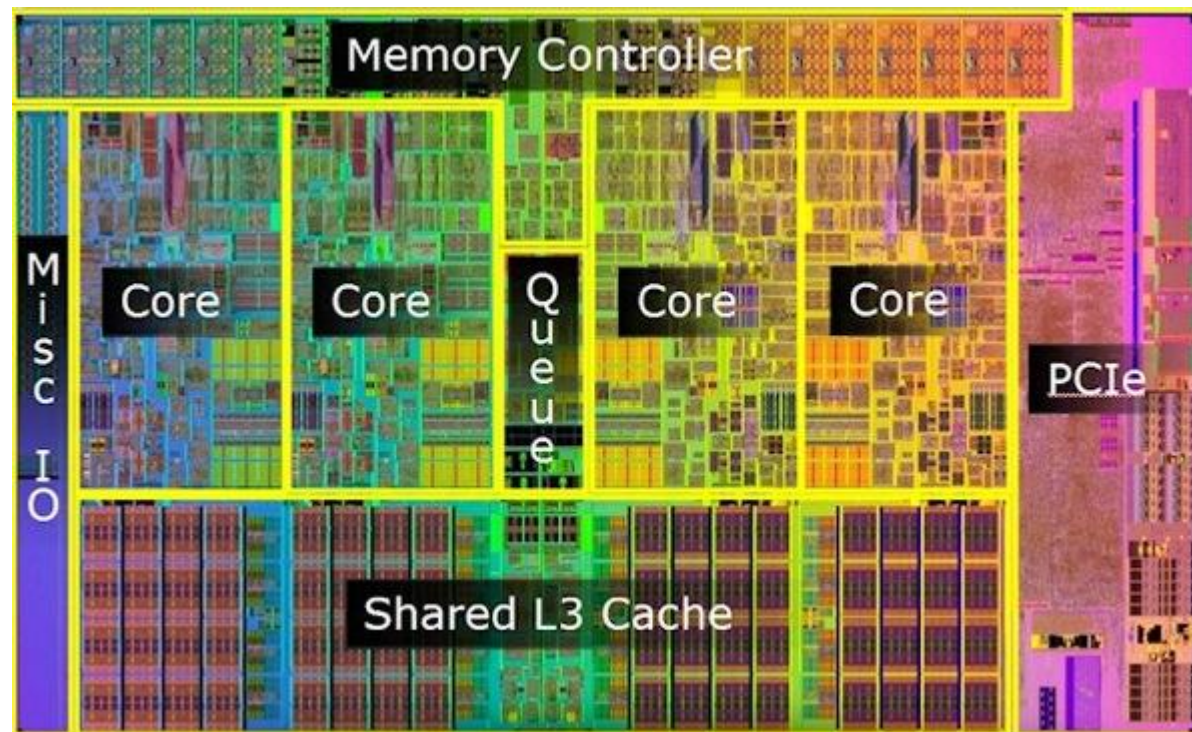
```
void matmulSSE(int mat1[N][N], int mat2[N][N], int result[N][N]) {
    for(int i = 0; i < N; ++i) {
        for(int j = 0; j < N; j+=4) { // vectorize over this loop
            __m128i vR = _mm_setzero_si128();
            for(int k = 0; k < N; k++) { // not this loop
                //result[i][j] += mat1[i][k] * mat2[k][j];
                __m128i vA = _mm_set1_epi32(mat1[i][k]); // load+broadcast
                __m128i vB = _mm_loadu_si128((__m128i*)&mat2[k][j]); // mat2[k][j+0..3]
                vR = _mm_add_epi32(vR, _mm_mullo_epi32(vA, vB));
            }
            _mm_storeu_si128((__m128i*)&result[i][j], vR);
        }
    }
}
```

# Intel x86's SSE : later versions

- SSE2: 64-bit floating point
- SSE3: Horizontal instructions
- SSE4: dot product, more integer instructions, population count,...
- AVX: 128-bit registers → 256-bit registers, 3-operand instructions
- AVX-512: → 512-bit registers

# Multicore parallelism

- “The free lunch is over”, Herb Sutter, 2005
  - In mid-2000, clock frequency had hit the power wall
  - Moore’s law was still going strong (the number of transistors doubles every 2 years)
  - → era of multicore
    - CPU cores have their own local state, run their own instruction evaluation loop
    - CPU cores can access the same memory space
      - → Share memory model



# Multi-threading

- The parallel capabilities of a multicore chip are usually made available to programmers through a **threading** abstraction.
  - A program can create multiple threads which can run concurrently on different cores of the same CPU
    - Hyperthreading: multiple thread can run in parallel on the same core
  - Threads can communicate to each other via share memory space
    - Concurrent control: mutex, semaphore, condition variable

# OpenMP: An API for writing multithreaded applications

- OpenMP:
  - Set of compiler directives and library routines for parallel programming
  - Greatly simplify writing multi-threaded programs
  - Standardizes for SMP practice
- Fork-join parallelism
  - Master thread spawns a teams of threads as needed
  - Parallelism added incrementally until performance goals are met

```
double A[1000];  
#pragma omp parallel num_threads(4)  
{  
    int ID = omp_get_thread_num();  
    pooh(ID,A);  
}
```

Example

# OpenMP: matrix multiplication

```
// multiply matrix A times matrix B, placing result in matrix C
void MatrixMult(int size, float * __restrict__ A,
                float * __restrict__ B, float * __restrict__ C) {
// split outer loop by threads
#pragma omp parallel for default(none) shared(A,B,C,size)
    for (int i = 0; i < size; ++i) {
        for (int j = 0; j < size; ++j) {
            float tmp = 0.; // private for each thread
            for (int k = 0; k < size; ++k) {
                tmp += A[i*size + k] * B[k*size + j];
            }
            C[i*size + j] = tmp; // update cell of C once
        }
    }
}
```

# Distributed computing

- Parallelism are made by running program on multiple **processes** on multiple computer (**nodes**) connected via an **interconnection network**
  - Can easily scale up by adding new nodes
  - Can be harm by failure on nodes and/or interconnection network
    - The term “parallel computing” normally refers to a model of parallel computing on dedicate system of reliable nodes and reliable interconnection network
      - High-level abstraction: MPI, PVM,...
    - The term “distributed computing” normally refers to a model of parallel computing on an unreliable multi-node system
      - High-level abstraction: MapReduce, Spark,...

# Message-passing model

- A parallel programming model in which processes communicate to each other via explicit message-passing operations





# Message-passing interface (MPI)

- Popular standard for parallel programming on parallel (reliable) system
  - Standard of naming, semantics, and return values of message-passing operations
  - Implemented by many parties: OpenMPI, LAM/MPI, MPICH, MVAPICH, IMPI,...
  - Programming languages: Fortran, C/C++, and more
- Single-program multiple-data model
  - The same program is launched on many nodes, each instance may work on its own portion of data, they communicate to each other via message-passing routines

#mpirun -np 4 ./prog

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <mpi.h>
4. int main(int argc, char* argv[])
5. {
6.     int size, rank;
7.     char  hostname[50];
8.     // Khoi tao tham so cho moi truong MPI
9.     MPI_Init(&argc, &argv);
10.    // Lay ve kich thuoc communicator
11.    MPI_Comm_size(MPI_COMM_WORLD, &size);
12.    // Lay ve so hieu rank
13.    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14.    // Lay ve hostname
15.    gethostname(hostname,50);
16.    printf("My rank is %d , %s\n",rank, hostname);
17.    MPI_Finalize();
18.    return 0;
19. }
```

# Message-passing interface: matrix multiplication

```
int main(int argc, char *argv[])
{
    int myrank, P, from, to, i, j, k;
    int tag = 666; /* any value will do */
    MPI_Status status;

    MPI_Init (&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank); /* who am i */
    MPI_Comm_size(MPI_COMM_WORLD, &P); /* number of processors */

    from = myrank * SIZE/P;
    to = (myrank+1) * SIZE/P;

    /* Process 0 fills the input matrices and broadcasts them to the rest */
    /* (actually, only the relevant stripe of A is sent to each process) */

    if (myrank==0) {
        fill_matrix(A);
        fill_matrix(B);
    }

    MPI_Bcast (B, SIZE*SIZE, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Scatter (A, SIZE*SIZE/P, MPI_INT, A[from], SIZE*SIZE/P, MPI_INT, 0, MPI_COMM_WORLD);

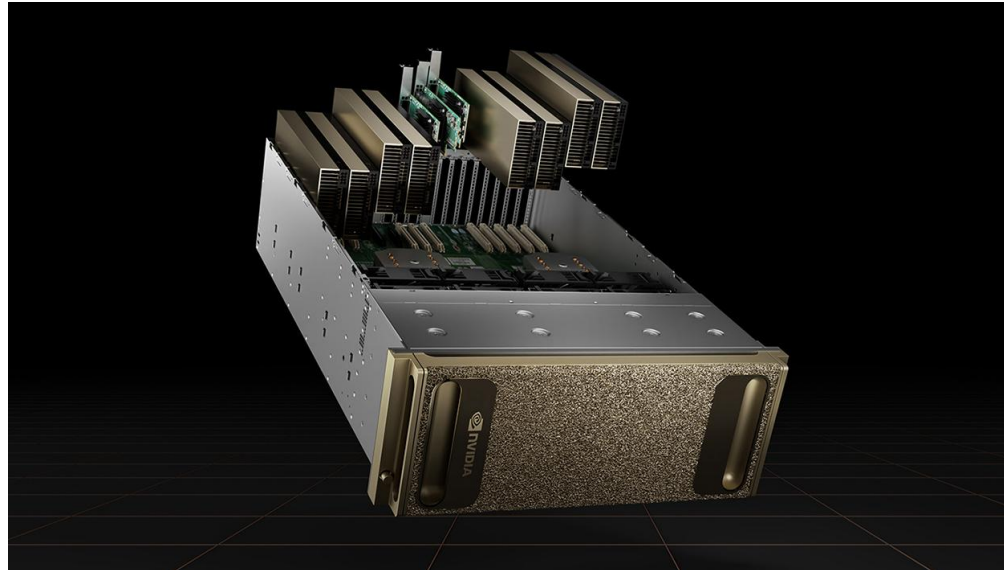
    printf("computing slice %d (from row %d to %d)\n", myrank, from, to-1);
    for (i=from; i<to; i++)
        for (j=0; j<SIZE; j++) {
            C[i][j]=0;
            for (k=0; k<SIZE; k++)
                C[i][j] += A[i][k]*B[k][j];
        }

    MPI_Gather (C[from], SIZE*SIZE/P, MPI_INT, C, SIZE*SIZE/P, MPI_INT, 0, MPI_COMM_WORLD);

    MPI_Finalize();
    return 0;
}
```

# Accelerated computing

- A portion of the data is offloaded to special devices (accelerators) and parallel computation is done there
- Accelerators: GPU, TPU, Intel Xeon Phi, FPGA,...



# NVIDIA's CUDA

- A programming environment for writing parallel code to run on model NVIDIA's GPUs
  - Support C/C++, Fortran
- Parallel code on GPU – kernel: same kernel processes multiple data on many GPU cores

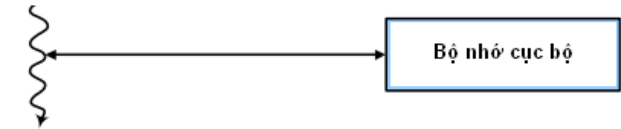
## CPU

```
...  
void inc_cpu(float *a, float b, int N) {  
    for (int idx = 0; idx < N; idx++)  
        a[idx] = a[idx] + b;  
}  
...  
void main() {  
    ...  
    inc_cpu(a, b, N);  
    ...  
}
```

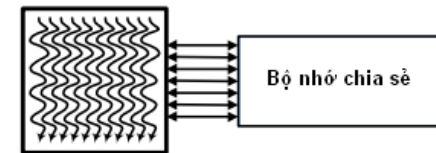
## CUDA

```
...  
__global__ void inc_gpu(float *a, float b, int N) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    if (idx < N)  
        a[idx] = a[idx] + b;  
}  
...  
void main() {  
    ...  
    dim3 dimBlock (blocksize);  
    dim3 dimGrid( ceil( N / (float)blocksize ) );  
    inc_gpu<<<dimGrid, dimBlock>>>(a, b, N);  
    ...  
}
```

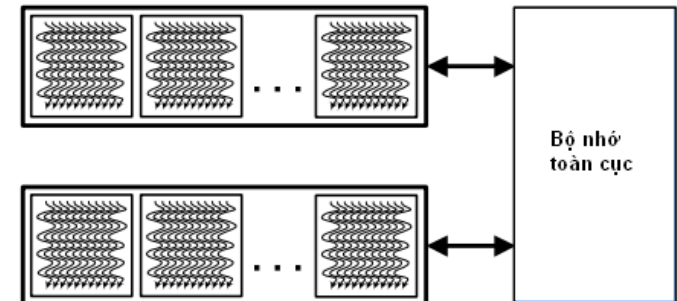
Luồng



Khối luồng



Lưới luồng



# NVIDIA's CUDA: matrix multiplication

## kernel

```
__global__ void matrixMultiplicationKernel(float* A, float* B, float* C, int N) {  
  
    int ROW = blockIdx.y*blockDim.y+threadIdx.y;  
    int COL = blockIdx.x*blockDim.x+threadIdx.x;  
  
    float tmpSum = 0;  
  
    if (ROW < N && COL < N) {  
        // each thread computes one element of the block sub-matrix  
        for (int i = 0; i < N; i++) {  
            tmpSum += A[ROW * N + i] * B[i * N + COL];  
        }  
    }  
    C[ROW * N + COL] = tmpSum;  
}
```

## Wrapper function

```
void matrixMultiplication(float *A, float *B, float *C, int N){  
  
    // declare the number of blocks per grid and the number of threads per block  
    // use 1 to 512 threads per block  
    dim3 threadsPerBlock(N, N);  
    dim3 blocksPerGrid(1, 1);  
    if (N*N > 512){  
        threadsPerBlock.x = 512;  
        threadsPerBlock.y = 512;  
        blocksPerGrid.x = ceil(double(N)/double(threadsPerBlock.x));  
        blocksPerGrid.y = ceil(double(N)/double(threadsPerBlock.y));  
    }  
  
    matrixMultiplicationKernel<<<blocksPerGrid,threadsPerBlock>>>(A, B, C, N);  
}
```

# Amdahl's law

$$S = \frac{1}{f + (1-f) / P}$$

**f** = Sequential part of the code.

**Ex.** **f** = 0.1                      assume **P** = 10 processes

$$S = \frac{1}{0.1 + (0.9) / 10}$$
$$= \frac{1}{0.1 + (0.09)} \cong 5$$

As **P**  $\longrightarrow \infty$                       **S**  $\longrightarrow 10$

Giá trị speedup tối đa có thể đạt được là 10



# HUST

# THANK YOU !